```
-> (params) { ... }
```

As a shortcut to:

```
lambda { |params| ... }
```

Why ->?



Let's start by getting something out of the way. Why ->? For compatibility across all the different source file encodings, Matz is restricted to using pure 7-bit ASCII for Ruby operators, and the choice of available characters is severely limited by the ambiguities inherent in the Ruby syntax. He felt that -> was (kind of) reminiscent of a Greek lambda character λ.

The parenthesis around the parameters are optional. Here are some examples:

```
proc1 = -> arg { puts "In proc1 with #{arg}" }
proc2 = -> arg1, arg2 { puts "In proc2 with #{arg1} and #{arg2}" }
proc3 = ->(arg1, arg2) { puts "In proc3 with #{arg1} and #{arg2}" }

proc1.call "ant"
proc2.call "bee", "cat"
proc3.call "dog", "elk"
```

produces:

```
In proc1 with ant
In proc2 with bee and cat
In proc3 with dog and elk
```

The -> form is more compact than using lambda and is especially useful when you want to pass one or more Proc objects to a method:

```
def my_if(condition, then_clause, else_clause)
  if condition
    then_clause.call
  else
    else_clause.call
  end
end

5.times do |val|
  my_if(
    val < 2,
    -> { puts "#{val} is small" },
    -> { puts "#{val} is big" }
  )
end
```

produces:

```
0 is small
1 is small
2 is big
3 is big
4 is big
```

One good reason to pass blocks to methods is that you can reevaluate the code in those blocks at any time.